

# PROYECTO FINAL

## Bases de Datos III

*BD III: PostgreSQL + MongoDB*

Modalidad: equipos de 2 personas

Cuatro escenarios disponibles

# 1. Presentación del proyecto

Este proyecto integra los conocimientos adquiridos durante el curso en un caso de aplicación realista. El objetivo no es simplemente que tu equipo construya un sistema funcional, sino que demuestre criterio técnico al diseñar, implementar y justificar una solución de persistencia que combina dos motores de bases de datos: PostgreSQL para los datos transaccionales y MongoDB para los datos flexibles y de alto volumen.

El proyecto se entrega como un enunciado de negocio. Tú y tu compañero deberán analizar los requerimientos, identificar las entidades del dominio, decidir qué información va a cada motor, diseñar el modelo, implementar los objetos de base de datos requeridos y exponer todo a través de una API mínima. La capacidad de modelar correctamente los datos es la habilidad más importante que se evaluará.

## Objetivos de aprendizaje

- Diseñar un modelo de datos relacional desde requerimientos de negocio, respetando normalización y aplicando reglas de integridad.
- Diseñar colecciones de MongoDB tomando decisiones conscientes sobre estructura de documentos, embebido vs. referencias y denormalización.
- Justificar técnicamente la elección entre PostgreSQL y MongoDB para cada tipo de información del sistema.
- Implementar vistas normales y materializadas, comprendiendo cuándo usar cada una y la estrategia de refresh apropiada.
- Implementar funciones y stored procedures con manejo robusto de errores, transacciones y concurrencia.
- Construir pipelines de aggregation que respondan a preguntas de negocio reales utilizando operadores adecuados.
- Analizar el rendimiento de consultas con EXPLAIN ANALYZE y justificar la creación de índices con datos.
- Diseñar e implementar una estrategia de respaldo y demostrar la capacidad de restaurar la base de datos ante un fallo.

## 2. Modalidad de trabajo

El proyecto se desarrolla en equipos de exactamente 2 personas. La conformación de equipos es libre, pero una vez registrados no se permiten cambios. Ambos integrantes son responsables del proyecto completo y ambos deben dominar todos los aspectos del trabajo entregado, ya que la defensa oral es individual y aleatoria: el catedrático puede preguntar a cualquiera de los dos sobre cualquier parte del sistema.

### Distribución del trabajo

Aunque se permite la distribución interna de tareas dentro del equipo, ambos integrantes deben estar familiarizados con el modelo completo, los objetos de PostgreSQL, los pipelines de MongoDB y las decisiones de diseño tomadas. Una distribución del tipo «yo hago Postgres y tú haces Mongo» es estratégicamente mala porque deja a cada integrante vulnerable en la mitad de la defensa.

#### Recomendación

Trabajen juntos en el diseño de la base de datos durante la primera semana. Es la parte más importante del proyecto y la que más impacto tiene en todo lo que viene después. A partir de ahí pueden distribuirse la implementación, pero ambos deben revisar y entender todo el código antes de la entrega.

### 3. Stack tecnológico obligatorio

El proyecto debe implementarse exclusivamente con el siguiente stack tecnológico. No se permiten ORMs ni abstracciones que oculten el SQL o los pipelines de MongoDB.

#### Motores de base de datos

- **PostgreSQL** como motor relacional, versión 14 o superior.
- **MongoDB** como motor NoSQL, versión 6 o superior. Se permite usar MongoDB Atlas o instalación local.

#### Capa de aplicación

- **Node.js** como runtime, versión 18 o superior.
- **Express** como framework HTTP.
- **pg (node-postgres)** como cliente de PostgreSQL. **No se permite usar Prisma, Sequelize, TypeORM ni ningún otro ORM relacional.** El motivo es pedagógico: el ORM oculta el SQL que estás aprendiendo a escribir.
- **Mongoose** como ODM de MongoDB. Sí está permitido porque no oculta el aggregation framework, que es donde está el aprendizaje principal de MongoDB en este curso.

#### Herramientas adicionales

- Git como sistema de control de versiones. El proyecto debe entregarse en un repositorio (puede ser privado en GitHub, GitLab o Bitbucket).
- Cualquier cliente gráfico para administración ( pgAdmin, MongoDB Compass) es libre.
- Para datos de prueba pueden usar herramientas como Faker.js o cualquier generador equivalente, siempre que los datos sean coherentes con el dominio elegido.

##### Sobre el uso de Claude Code u otros asistentes de IA

Está permitido usar Claude Code o cualquier asistente de IA para acelerar el desarrollo de la capa de aplicación (Express, conexiones, rutas). Sin embargo, los objetos de base de datos (vistas, funciones, stored procedures, pipelines de aggregation) deben ser escritos y comentados por ti y tu compañero. Más detalles en la sección «Uso responsable de IA».

## 5. Requisitos técnicos generales

Estos requisitos aplican a las cuatro opciones de proyecto. Los detalles específicos de cada escenario se encuentran en la Parte II de este documento.

### 5.1 Diseño de la base de datos

El diseño es la parte central del proyecto. Tu equipo debe entregar:

- Diagrama Entidad-Relación (ER) del modelo PostgreSQL. Puede usarse cualquier herramienta (dbdiagram.io, draw.io, Lucidchart, pgModeler).
- Diagrama o documento que describa la estructura de cada colección de MongoDB, especificando tipos de datos, embebidos, referencias a Postgres y decisiones de denormalización.
- Documento de decisiones de diseño (entre 2 y 3 páginas) que justifique: por qué se eligieron esas entidades y no otras, qué información va a Postgres y qué a Mongo y por qué, qué reglas de negocio se aplican a nivel de schema (FK, CHECK, UNIQUE) y cuáles a nivel de procedure, qué decisiones se tomaron en MongoDB.
- Modelo relacional normalizado al menos hasta la **3ª Forma Normal**. Cualquier desnormalización debe estar justificada.
- Definición de índices apropiados en ambos motores, con justificación técnica de cada uno.

### 5.2 Objetos de PostgreSQL

Como mínimo, tu equipo debe implementar:

- **2 vistas normales** que respondan a alguna de las consultas requeridas del escenario.
- **2 vistas materializadas** con su estrategia de refresh documentada.
- **3 funciones**, al menos una escalar y al menos una que retorne *TABLE*.
- **2 stored procedures** que implementen las operaciones críticas del escenario, con manejo robusto de errores mediante bloques *BEGIN/EXCEPTION/ROLLBACK* y manejo de concurrencia donde aplique.
- **Índices justificados** en las tablas con análisis de impacto medido.

### 5.3 Objetos de MongoDB

- **1 función JavaScript reutilizable** para procesamiento o normalización de datos antes de inserción.
- **4 pipelines de aggregation** que respondan a las consultas requeridas del escenario.
- Al menos uno de los pipelines debe utilizar *\$facet* para devolver múltiples resultados en una sola ejecución.

## 5.4 Capa de aplicación

API REST mínima en Node.js + Express que exponga endpoints para invocar las operaciones críticas y obtener los reportes principales. La API es un medio para demostrar el funcionamiento de los objetos de base de datos, no el objetivo del proyecto. No se evaluará la calidad del frontend ni patrones avanzados de arquitectura, sino que los endpoints invoquen correctamente a los objetos de base de datos creados.

Como mínimo, la API debe exponer:

- Un endpoint para cada operación crítica (POST que invoque el stored procedure correspondiente).
- Un endpoint para cada reporte gerencial (GET que consulte la vista materializada o pipeline de aggregation correspondiente).
- Un endpoint para inserción de datos en MongoDB que utilice la función JavaScript de procesamiento.

## 5.5 Análisis de performance

Reporte técnico de entre 3 y 5 páginas que incluya:

- Comparación con **EXPLAIN ANALYZE** de al menos 2 queries en versión optimizada (con vista materializada y/o índices) vs. versión no optimizada. Deben incluirse los planes de ejecución completos y los tiempos medidos.
- Justificación técnica de cada índice creado, con mediciones del impacto antes y después de su creación.
- Estrategia de refresh de las vistas materializadas, justificando frecuencia, momento del día y costo del refresh.

## 5.6 Estrategia de respaldo

- Script de backup full de PostgreSQL (puede usarse pg\_dump).
- Script de backup incremental usando WAL archiving o herramienta equivalente.
- Política de retención documentada (cuántos backups se conservan, por cuánto tiempo, con qué frecuencia).
- **Demostración de restauración** sobre una base limpia, con validación de integridad. Esta demostración debe incluirse en la defensa final.

## 5.7 Documentación

- README técnico con instrucciones claras para que cualquier persona pueda levantar el sistema desde cero (creación de bases, ejecución de scripts, instalación de dependencias, levantamiento de la API, ejecución de datos de prueba).
- Documento de decisiones de diseño descrito en la sección 5.1.
- Reporte de performance descrito en la sección 5.5.
- Bitácora de uso de IA (descrita en la sección 7).

## 6. Defensa oral

La defensa oral es el componente con mayor peso en la evaluación y es el filtro principal para validar que el conocimiento es real y no copiado de un asistente de IA. La defensa se realiza en la entrega final con cita programada por equipo.

### Formato

- Duración: entre 5 y 10 minutos por equipo.
- Modalidad: presencial frente al catedrático, ambos integrantes presentes.
- Material a presentar: el equipo demuestra el sistema funcionando en vivo, ejecutando consultas, invocando procedures y mostrando reportes.
- **Preguntas individuales y aleatorias:** el catedrático puede preguntar a cualquiera de los dos integrantes sobre cualquier parte del sistema. La calificación de la defensa puede ser distinta para cada integrante si uno demuestra mayor dominio que el otro.

### Temas que pueden ser preguntados

- Justificación de cualquier decisión de diseño: por qué esa entidad, por qué esa relación, por qué esa colección, por qué embebido o referenciado.
- Comportamiento de cualquier vista, función o stored procedure: qué hace, cómo lo hace, qué pasa si se invoca con datos inválidos.
- Lógica de cualquier pipeline de aggregation: qué hace cada etapa, por qué se eligieron esos operadores, qué pasaría si se reordenan.
- Resultados del análisis de performance: por qué esa query es más rápida con índice, qué muestra el plan de ejecución.
- Demostración en vivo de la restauración del backup.

#### Cómo prepararse

La mejor preparación es escribir tú mismo cada línea de código y entender por qué está ahí. Si tu equipo usó IA para generar parte del código, dediquen tiempo a estudiar lo generado, modificarlo, romperlo y arreglarlo. La defensa no se aprueba con código bonito; se aprueba con conocimiento real.



## 7. Uso responsable de IA

Se permite —y se recomienda— el uso de Claude Code, ChatGPT, GitHub Copilot u otros asistentes de IA en este proyecto. Estas herramientas son parte de la realidad profesional de la ingeniería de software hoy, y no tiene sentido prohibirlas. Sin embargo, el uso debe ser estratégico y transparente.

### Qué se permite

- Usar IA para generar código de la capa de aplicación: rutas de Express, conexiones, manejo de errores HTTP, validaciones.
- Usar IA para explicarte conceptos que no entiendas, depurar errores, sugerir índices o estrategias.
- Usar IA para generar datos de prueba (faker, scripts de seed).
- Usar IA para revisar tu código y sugerir mejoras.

### Qué no se permite

- Pedir a la IA que escriba **directamente** los objetos de base de datos: vistas, funciones, stored procedures y pipelines de aggregation deben ser escritos y comprendidos por ti. Puedes pedir explicaciones o ejemplos, pero el código final debe ser tuyo.
- Entregar código que tú no entiendas. Si lo entregas, asume que en la defensa te van a preguntar exactamente sobre eso.
- Entregar código sin verificar que funciona. La IA se equivoca; tu deber es probar.

### Bitácora de uso de IA

Como parte de la entrega final, debes incluir una bitácora (puede ser un archivo Markdown) donde documentes con honestidad:

- Qué le pediste a la IA en cada caso (no necesitas pegar todos los prompts; basta con un resumen por categoría: «pedí ayuda para configurar la conexión a Postgres», «pedí ejemplos de uso de \$facet», etc.).
- Qué partes del código generado modificaste manualmente y por qué.
- Qué errores cometió la IA que tuviste que corregir.

#### Por qué pedimos la bitácora

No es para vigilarte ni para penalizarte; es porque saber dónde y cómo usar IA es una habilidad profesional. Quienes la usan bien entregan más rápido y aprenden más. Quienes la usan mal

entregan código que no entienden y se caen en la defensa. La bitácora te obliga a reflexionar sobre cuál de los dos casos es el tuyo.

## 8. Rúbrica de evaluación

La calificación final del proyecto se compone de los siguientes elementos:

| Componente   | Peso | Descripción breve                                                           |
|--------------|------|-----------------------------------------------------------------------------|
| Defensa oral | 100% | Capacidad de explicar y justificar técnicamente cada decisión del proyecto. |

### Criterios de aprobación

- Las entregas parciales son requisito para la entrega final. Un equipo que no presente alguna de las entregas parciales no podrá presentar el proyecto final.
- La defensa oral es individual aleatoria. Si uno de los integrantes no puede responder preguntas básicas sobre el proyecto, su nota personal puede verse penalizada.
- La presencia de plagio entre equipos invalida automáticamente el proyecto para los equipos involucrados.

# PARTE II

## Escenarios disponibles

*A continuación se presentan los cuatro escenarios entre los que tu equipo debe elegir uno para desarrollar el proyecto.*

*La elección debe registrarse con el catedrático antes del fin de la primera semana.*

**Opción A — Clínica médica privada**

**Opción B — Hotel**

**Opción C — Plataforma de cursos online**

**Opción D — Delivery de comida**

## Opción A — Clínica médica privada

### Resumen del escenario

Sistema integral para una clínica privada que maneja agendamiento de citas, atención médica con registro clínico variable según especialidad, facturación con pagos parciales y reportes gerenciales. La persistencia se justifica porque lo financiero requiere ACID estricto, mientras que los historiales clínicos varían sustancialmente entre especialidades y son ideales para schemaless.

### 1. Descripción del negocio

Una clínica médica privada de la ciudad ofrece servicios de consulta general y atención en distintas especialidades médicas. La operación involucra a tres tipos de actores: el personal de recepción, que agenda citas y emite facturas; los médicos, que atienden pacientes y registran información clínica; y los pacientes, que solicitan atención y pagan los servicios recibidos.

El flujo típico es el siguiente: un paciente llama o llega a la clínica para solicitar una cita con un médico de una especialidad determinada. Recepción verifica la disponibilidad del médico en la fecha y hora deseadas y agenda la cita. El día de la cita, el médico atiende al paciente y registra toda la información clínica relevante: motivo de la consulta, signos vitales, diagnósticos, medicamentos recetados y exámenes solicitados. La información clínica registrada **varía sustancialmente entre especialidades** — un cardiólogo no registra lo mismo que un dermatólogo o un pediatra. Al finalizar la consulta, recepción genera la factura por los servicios prestados, que el paciente puede pagar de una sola vez o en pagos parciales.

La clínica también requiere mantener un registro de auditoría sobre las acciones realizadas en el sistema (quién agendó qué cita, quién registró qué pago, etc.) y necesita generar reportes gerenciales periódicos para análisis financiero y clínico.

### 2. Requerimientos funcionales

**RF-01.** El sistema debe permitir registrar y gestionar el catálogo de especialidades médicas que ofrece la clínica.

**RF-02.** El sistema debe permitir registrar médicos, asociarlos a una especialidad y definir sus horarios de atención semanales.

**RF-03.** El sistema debe permitir registrar pacientes con sus datos personales y de contacto.

**RF-04.** El sistema debe permitir agendar citas para un paciente con un médico específico en una fecha y hora determinadas, validando que el médico tenga disponibilidad en ese horario y no tenga otra cita programada al mismo tiempo.

**RF-05.** El sistema debe permitir consultar la disponibilidad de un médico en una fecha dada, mostrando los horarios libres dentro de su jornada de atención.

**RF-06.** El sistema debe permitir cambiar el estado de una cita a lo largo de su ciclo de vida (programada, confirmada, atendida, cancelada, no asistió).

**RF-07.** Al atender una cita, el médico debe poder registrar información clínica detallada de la consulta. Esta información incluye al menos: motivo de la consulta, signos vitales del paciente, diagnósticos identificados, medicamentos recetados, exámenes solicitados y notas adicionales. **La estructura específica de esta información puede variar según la especialidad y el caso clínico.**

**RF-08.** El sistema debe permitir consultar el historial clínico completo de un paciente, mostrando todas sus consultas anteriores en orden cronológico.

**RF-09.** El sistema debe mantener un catálogo de servicios facturables (consultas, procedimientos, exámenes) con su precio.

**RF-10.** El sistema debe permitir generar facturas por los servicios prestados al paciente. Una factura puede incluir uno o varios servicios.

**RF-11.** El sistema debe permitir registrar pagos asociados a una factura. Una factura puede ser pagada en uno o varios pagos parciales hasta cubrir el total.

**RF-12.** El sistema debe llevar un registro de auditoría de las operaciones críticas realizadas (creación de cita, cancelación, registro de pago, etc.), incluyendo quién la realizó, cuándo y los detalles relevantes.

### 3. Reglas de negocio

**RN-01.** Un médico no puede tener dos citas atendidas o programadas en el mismo horario.

**RN-02.** Una cita solo puede agendarse dentro de los horarios de atención definidos para el médico.

**RN-03.** Un paciente no puede tener más de una cita activa con el mismo médico en el mismo día.

**RN-04.** El monto total de los pagos asociados a una factura no puede exceder el total de la factura.

**RN-05.** No se pueden registrar pagos sobre facturas anuladas.

**RN-06.** Una factura cambia automáticamente de estado según los pagos recibidos: pendiente (sin pagos), pagada parcial (pagos > 0 pero < total), pagada (pagos = total).

**RN-07.** Una cita cancelada debe registrar el motivo de la cancelación.

**RN-08.** El historial clínico de una consulta solo puede registrarse para citas en estado «atendida».

### 4. Reportes y consultas requeridas

El sistema debe poder generar las siguientes consultas y reportes. **Tu equipo decidirá qué mecanismo usar para implementar cada uno** (consulta directa, vista normal, vista materializada, función o pipeline de aggregation), justificando su decisión.

**RC-01.** Agenda diaria: todas las citas de un día dado, con paciente, médico, especialidad, hora y estado. Consulta de uso intensivo por recepción.

**RC-02.** Facturas pendientes: facturas con saldo pendiente, ordenadas por antigüedad, con datos del paciente y monto adeudado.

**RC-03.** Facturación mensual: total facturado, cobrado y saldo pendiente, agrupado por mes y especialidad. Reporte gerencial consultado frecuentemente, sin necesidad de información en tiempo real.

**RC-04.** Ranking trimestral de médicos: médicos ordenados por número de citas atendidas y monto facturado en el último trimestre.

**RC-05.** Disponibilidad de médico: dado un médico y una fecha, los horarios libres de su jornada.

**RC-06.** Saldo de paciente: total adeudado por un paciente, sumando todas sus facturas pendientes.

**RC-07.** Top 5 diagnósticos más frecuentes del último trimestre, agrupados por especialidad.

**RC-08.** Medicamentos más recetados por especialidad, con cantidad de prescripciones y porcentaje del total.

**RC-09.** Análisis de signos vitales por grupo etario: promedio de presión arterial, frecuencia cardiaca y otros indicadores, agrupados por rangos de edad de los pacientes.

**RC-10.** Tiempo promedio entre consultas por paciente (días entre una consulta y la siguiente), para identificar pacientes con seguimiento frecuente.

## 5. Operaciones críticas

Las siguientes operaciones involucran múltiples pasos y deben ejecutarse de forma **atómica** (todo o nada). Tu equipo debe implementarlas de manera que un fallo en cualquier paso revierta todos los cambios anteriores.

**OC-01. Registro de pago.** Al recibir un pago, el sistema debe: (a) validar que la factura exista y no esté anulada, (b) validar que el monto sea positivo y no exceda el saldo pendiente, (c) registrar el pago, (d) actualizar el estado de la factura según corresponda, (e) registrar la operación en el log de auditoría. Si cualquier paso falla, ningún cambio debe persistir.

**OC-02. Cancelación de cita.** Al cancelar una cita, el sistema debe: (a) validar que la cita exista y no esté ya atendida, (b) cambiar su estado a cancelada con el motivo proporcionado, (c) liberar el horario para que pueda agendarse otra cita, (d) registrar la operación en el log de auditoría.

## 6. Volumen mínimo de datos de prueba

Tu equipo debe cargar al menos los siguientes volúmenes para que las consultas y reportes muestren resultados significativos:

| Entidad                         | Cantidad mínima                                                |
|---------------------------------|----------------------------------------------------------------|
| Especialidades                  | 5                                                              |
| Médicos                         | 10 (al menos 2 por especialidad)                               |
| Pacientes                       | 30 con edades variadas                                         |
| Citas                           | 200 distribuidas en los últimos 6 meses, con mezcla de estados |
| Historiales clínicos en MongoDB | 150 (uno por cada cita atendida)                               |
| Facturas                        | 100 con sus detalles                                           |
| Pagos                           | 80 con mezcla de pagos completos y parciales                   |
| Entradas de log de auditoría    | 500 o más                                                      |



## Opción B — Hotel

### Resumen del escenario

Sistema integral para un hotel de mediano tamaño que maneja reservas, estadía con consumos extras, facturación con pagos parciales, perfiles personalizados de huéspedes, incidencias y reseñas. La persistencia políglota se justifica porque las reservas requieren bloqueo y ACID estricto para evitar doble venta de habitaciones, mientras que los perfiles, incidencias y reseñas son datos flexibles ideales para schemaless.

### 1. Descripción del negocio

Un hotel de mediano tamaño opera con habitaciones distribuidas en varios pisos, agrupadas en distintas categorías que se diferencian por capacidad, comodidades y precio. El negocio gira alrededor de tres flujos principales: **reservas, estadía y facturación**.

El flujo típico inicia cuando un huésped solicita una reserva para fechas determinadas, indicando el tipo de habitación que desea y la cantidad de personas que se hospedarán. La recepción verifica la disponibilidad de habitaciones que cumplan con los requisitos en el rango de fechas solicitado y, si hay disponibilidad, confirma la reserva. **El sistema debe garantizar que dos huéspedes no puedan reservar la misma habitación para fechas que se solapen** — este es uno de los problemas operativos más críticos del negocio.

Al llegar el huésped, recepción ejecuta el check-in y le asigna formalmente la habitación. Durante su estadía, el huésped puede consumir servicios extras del hotel (minibar, lavandería, spa, restaurante, transporte), los cuales se cargan a su cuenta. Al momento del check-out, se cierra la factura sumando el hospedaje y todos los consumos, y el huésped paga el total. El pago puede realizarse de una sola vez o en pagos parciales, especialmente en el caso de huéspedes corporativos que abonan parte al reservar y el resto al salir.

El hotel también necesita mantener información detallada de sus huéspedes recurrentes — preferencias personales (tipo de almohada, temperatura preferida, vista deseada), restricciones alimentarias, alergias, idiomas, peticiones especiales, programa de lealtad — que **varía considerablemente entre huéspedes** y crece con cada estadía. Adicionalmente, durante las estadías pueden ocurrir incidencias (quejas, solicitudes especiales, observaciones) que deben registrarse junto con las acciones tomadas para resolverlas. Al finalizar su estadía, los huéspedes pueden dejar reseñas evaluando distintos aspectos del servicio.

### 2. Requerimientos funcionales

**RF-01.** El sistema debe permitir gestionar el catálogo de tipos de habitación, definiendo capacidad de personas, comodidades y precio por noche.

**RF-02.** El sistema debe permitir registrar las habitaciones físicas del hotel, asociadas a un tipo, ubicadas en un piso específico, con un estado operativo (disponible, ocupada, mantenimiento, en limpieza).

**RF-03.** El sistema debe permitir registrar huéspedes con sus datos personales y de contacto.

**RF-04.** El sistema debe permitir consultar la disponibilidad de habitaciones en un rango de fechas, filtrando por tipo de habitación y capacidad requerida. El sistema debe excluir habitaciones que tengan reservas confirmadas que se solapen con las fechas solicitadas.

**RF-05.** El sistema debe permitir crear reservas para un huésped, en una habitación específica, para un rango de fechas. La reserva debe calcular automáticamente el total estimado.

**RF-06.** El sistema debe permitir gestionar el ciclo de vida de una reserva (pendiente, confirmada, en curso, completada, cancelada, no show).

**RF-07.** El sistema debe mantener un catálogo de servicios extras facturables, organizados por categoría (minibar, lavandería, spa, restaurante, transporte).

**RF-08.** El sistema debe permitir registrar consumos de servicios extras durante la estadía, asociados a una reserva activa.

**RF-09.** El sistema debe permitir ejecutar el check-out de una reserva, generando la factura final con el total de hospedaje más todos los consumos extras.

**RF-10.** El sistema debe permitir registrar pagos asociados a una factura. Una factura puede ser pagada en uno o varios pagos parciales hasta cubrir el total.

**RF-11.** El sistema debe permitir gestionar información detallada del perfil de cada huésped, incluyendo preferencias, alergias, restricciones, idiomas, peticiones especiales y datos del programa de lealtad. **La estructura específica de esta información puede variar entre huéspedes.**

**RF-12.** El sistema debe permitir registrar incidencias ocurridas durante una estadía (quejas, solicitudes, observaciones), incluyendo descripción, categoría, estado de resolución y las acciones tomadas con sus responsables.

**RF-13.** El sistema debe permitir a los huéspedes registrar reseñas al finalizar su estadía, evaluando múltiples aspectos del servicio (limpieza, atención, ubicación, confort, relación calidad-precio) e incluyendo comentarios libres y aspectos destacables o mejorables.

### 3. Reglas de negocio

**RN-01.** Una habitación no puede tener dos reservas confirmadas que se solapen en fechas.

**RN-02.** Una reserva no puede crearse sobre una habitación que esté en estado de mantenimiento.

**RN-03.** El número de huéspedes de una reserva no puede exceder la capacidad del tipo de habitación reservado.

**RN-04.** Solo se pueden registrar consumos sobre reservas en estado «en curso».

**RN-05.** Una reserva solo puede pasar a estado «completada» después de un check-out exitoso.

**RN-06.** El monto total de los pagos asociados a una factura no puede exceder el total de la factura.

**RN-07.** Una factura cambia automáticamente de estado según los pagos recibidos: pendiente, pagada parcial, pagada.

**RN-08.** Una habitación que sale de check-out queda automáticamente en estado «limpieza» antes de poder ser asignada a una nueva reserva.

**RN-09.** Solo pueden registrar reseña los huéspedes cuya reserva esté en estado «completada».

## 4. Reportes y consultas requeridas

El sistema debe poder generar las siguientes consultas y reportes. **Tu equipo decidirá qué mecanismo usar para implementar cada uno** (consulta directa, vista normal, vista materializada, función o pipeline de aggregation), justificando su decisión.

**RC-01.** Disponibilidad inmediata: habitaciones disponibles en este momento, con su tipo, capacidad y precio. Consulta de uso intensivo en recepción.

**RC-02.** Huéspedes hospedados actualmente: huéspedes en estadía en curso, con número de habitación, fechas de check-in y check-out estimado, días de estadía transcurridos.

**RC-03.** Disponibilidad en rango: dadas dos fechas y un tipo de habitación, listar las habitaciones libres en ese rango.

**RC-04.** Cálculo de total de reserva: dado un rango de fechas y una habitación, calcular el total a cobrar.

**RC-05.** Saldo de una reserva: total facturado menos pagos efectuados.

**RC-06.** Ocupación mensual: porcentaje de ocupación, ingresos por hospedaje, ingresos por consumos extras, agrupado por mes y tipo de habitación. Reporte gerencial frecuente, sin necesidad de información en tiempo real.

**RC-07.** Top huéspedes anuales: huéspedes con más estadías y mayor gasto en el último año, útil para programa de lealtad.

**RC-08.** Análisis de satisfacción por trimestre: promedio de cada métrica de calificación de las reseñas (limpieza, atención, ubicación, confort, calidad-precio), agrupado por trimestre, para identificar tendencias.

**RC-09.** Top aspectos mejorables: aspectos más mencionados por los huéspedes como mejorables, con tendencia temporal para detectar problemas recurrentes.

**RC-10.** Análisis de incidencias: cantidad de incidencias por categoría, tasa de resolución y tiempo promedio de resolución (desde el reporte hasta la última acción tomada).

**RC-11.** Perfil del huésped recurrente: análisis agregado de preferencias y características de los huéspedes con más de 3 estadías (preferencias más comunes, alergias frecuentes, idiomas predominantes).

## 5. Operaciones críticas

Las siguientes operaciones involucran múltiples pasos y deben ejecutarse de forma **atómica** (todo o nada). Tu equipo debe implementarlas de manera que un fallo en cualquier paso revierta todos los cambios anteriores.

**OC-01. Realización de reserva.** Al crear una reserva, el sistema debe: (a) validar que la habitación exista y no esté en mantenimiento, (b) **validar que no exista solape con otra reserva confirmada para la misma habitación, manejando correctamente la concurrencia** (dos solicitudes simultáneas para la misma habitación no deben poder crear ambas la reserva), (c) validar que la cantidad de huéspedes no exceda la capacidad, (d) calcular el total estimado, (e) crear la reserva en estado confirmada, (f) generar la factura preliminar en estado pendiente, (g) registrar la operación en el log de auditoría. Si cualquier paso falla, ningún cambio debe persistir.

**OC-02. Check-out de reserva.** Al ejecutar el check-out, el sistema debe: (a) validar que la reserva esté en estado «en curso», (b) calcular el total final de la factura sumando hospedaje y todos los consumos extras registrados, (c) cerrar la factura con el total final, (d) registrar el pago recibido y actualizar el estado de la factura, (e) cambiar el estado de la reserva a «completada», (f) cambiar el estado de la habitación a «limpieza», (g) registrar la operación en el log de auditoría.

## 6. Volumen mínimo de datos de prueba

Tu equipo debe cargar al menos los siguientes volúmenes para que las consultas y reportes muestren resultados significativos:

| Entidad             | Cantidad mínima                                                                           |
|---------------------|-------------------------------------------------------------------------------------------|
| Tipos de habitación | 4                                                                                         |
| Habitaciones        | 50 distribuidas en al menos 5 pisos                                                       |
| Huéspedes           | 80 con perfiles variados                                                                  |
| Reservas            | 250 distribuidas en los últimos 12 meses, con mezcla de estados (incluir varias en curso) |

| Entidad                                     | Cantidad mínima                             |
|---------------------------------------------|---------------------------------------------|
| Perfiles detallados de huéspedes en MongoDB | 60                                          |
| Consumos de servicios extras                | 800 o más                                   |
| Facturas con sus pagos                      | 200 (mezcla de pagos completos y parciales) |
| Incidencias en MongoDB                      | 40                                          |
| Reseñas en MongoDB                          | 120                                         |

## Opción C — Plataforma de cursos online

### Resumen del escenario

Plataforma de educación en línea estilo Udemy o Coursera, donde instructores publican cursos y estudiantes los consumen. La persistencia políglota se justifica porque las inscripciones, pagos y certificados requieren ACID estricto, mientras que el progreso de los estudiantes (alto volumen), las respuestas a cuestionarios (estructura variable) y los foros jerárquicos son ideales para MongoDB.

### 1. Descripción del negocio

Una plataforma de educación en línea ofrece cursos producidos por instructores especializados a estudiantes de todo el mundo. El modelo de negocio es similar al de plataformas como Udemy o Coursera, pero a menor escala. Hay tres actores principales: **instructores**, que crean y publican cursos; **estudiantes**, que se inscriben y consumen el contenido; y **administradores**, que gestionan la plataforma y supervisan métricas.

Cada curso está organizado en **módulos**, y cada módulo contiene una secuencia de **lecciones** (videos, lecturas, cuestionarios). Los estudiantes pueden inscribirse en cualquier curso pagando su precio, después de lo cual obtienen acceso permanente al contenido. La plataforma cobra una comisión sobre cada inscripción, y el resto se acredita al instructor del curso.

A medida que un estudiante avanza, la plataforma debe **registrar de forma detallada su progreso**: qué lecciones ha visto, cuánto tiempo dedicó a cada una, en qué punto del video quedó, cuándo completó cada módulo. Cuando un estudiante toma un cuestionario, se almacenan sus respuestas y la calificación obtenida — y los cuestionarios pueden tener distintos tipos de pregunta (selección única, múltiple, verdadero/falso, respuesta corta), cada una con estructura propia.

Los estudiantes pueden además interactuar a través de **foros de discusión** asociados a cada curso, publicando preguntas y respuestas, recibiendo «me gusta» y respondiendo entre ellos en hilos de profundidad variable. Al completar un curso, el estudiante puede dejar una **calificación y reseña** del instructor, y obtiene un certificado que queda registrado.

La plataforma necesita generar reportes financieros (ingresos, comisiones, pagos a instructores) y reportes pedagógicos (tasas de finalización, lecciones donde los estudiantes más abandonan, cursos con mejor desempeño).

### 2. Requerimientos funcionales

**RF-01.** El sistema debe permitir registrar instructores con sus datos personales, biografía profesional y datos para liquidación de pagos.

**RF-02.** El sistema debe permitir registrar estudiantes con sus datos personales y de contacto.

**RF-03.** El sistema debe permitir gestionar el catálogo de categorías de cursos (programación, diseño, negocios, idiomas, etc.).

**RF-04.** El sistema debe permitir a los instructores crear cursos, asignándoles un título, descripción, categoría, precio y nivel (principiante, intermedio, avanzado). Un curso puede estar en estado borrador, publicado, archivado.

**RF-05.** El sistema debe permitir organizar el contenido de un curso en **módulos**, y cada módulo en **lecciones** ordenadas. Cada lección tiene un tipo (video, lectura, cuestionario) y una duración estimada.

**RF-06.** El sistema debe permitir inscribir a un estudiante en un curso, registrando el pago correspondiente y la comisión que retiene la plataforma.

**RF-07.** El sistema debe permitir registrar el avance del estudiante en cada lección consumida: cuándo la inició, cuándo la completó, tiempo dedicado, y en el caso de videos, el último punto visualizado. **El detalle del progreso es de alto volumen y debe almacenarse de forma eficiente.**

**RF-08.** El sistema debe permitir registrar las respuestas que un estudiante da a un cuestionario, incluyendo cada respuesta individual y la calificación obtenida. **Las preguntas pueden ser de distintos tipos (selección única, múltiple, verdadero/falso, respuesta corta), por lo que la estructura de las respuestas varía.**

**RF-09.** El sistema debe permitir a los estudiantes participar en los foros de discusión de cada curso, creando publicaciones, respondiendo a publicaciones existentes en hilos de profundidad variable, y reaccionando con «me gusta». **El contenido del foro es jerárquico y de estructura variable.**

**RF-10.** El sistema debe permitir a un estudiante calificar y reseñar un curso una vez completado, evaluando múltiples aspectos (calidad del contenido, claridad del instructor, dificultad apropiada, valor por el precio).

**RF-11.** El sistema debe permitir emitir certificados a los estudiantes que completen el 100% de las lecciones de un curso, registrando fecha de emisión y un código único de verificación.

**RF-12.** El sistema debe llevar un registro de auditoría sobre las operaciones críticas (inscripciones, pagos, emisión de certificados, etc.).

### 3. Reglas de negocio

**RN-01.** Un estudiante no puede inscribirse dos veces al mismo curso.

**RN-02.** Solo se pueden inscribir estudiantes en cursos que estén en estado «publicado».

**RN-03.** El precio de la inscripción debe corresponder al precio vigente del curso al momento de la inscripción (la inscripción debe registrar el monto pagado, aunque el curso después cambie de precio).

**RN-04.** La comisión de la plataforma es un porcentaje fijo configurable (sugerido 30%) sobre el precio del curso. El resto se acredita al instructor.

**RN-05.** Solo pueden calificar y reseñar un curso los estudiantes que estén inscritos y hayan completado al menos el 80% de las lecciones.

**RN-06.** Un certificado solo puede emitirse cuando el estudiante ha completado el 100% de las lecciones del curso.

**RN-07.** Una lección no puede pertenecer a dos módulos distintos.

**RN-08.** Un curso no puede pasar a estado «archivado» si tiene estudiantes activos (con inscripciones recientes y progreso en curso).

**RN-09.** El progreso de un estudiante en una lección no puede registrarse si el estudiante no está inscrito en el curso correspondiente.

## 4. Reportes y consultas requeridas

El sistema debe poder generar las siguientes consultas y reportes. **Tu equipo decidirá qué mecanismo usar para implementar cada uno** (consulta directa, vista normal, vista materializada, función o pipeline de aggregation), justificando su decisión.

**RC-01.** Catálogo de cursos publicados: cursos visibles para los estudiantes con título, instructor, categoría, precio, calificación promedio y cantidad de estudiantes inscritos.

**RC-02.** Cursos del estudiante: lista de cursos en los que un estudiante está inscrito, con porcentaje de avance, fecha de inscripción y estado (en curso, completado).

**RC-03.** Cálculo de avance: porcentaje de avance de un estudiante en un curso (lecciones completadas / lecciones totales).

**RC-04.** Ingresos por instructor: total acreditado a un instructor en un período dado, descontando la comisión de la plataforma.

**RC-05.** Reporte mensual de ingresos: ingresos brutos, comisión de la plataforma, monto a liquidar a instructores, agrupado por mes y categoría de curso. Reporte gerencial frecuente.

**RC-06.** Top 10 cursos más vendidos del trimestre: cursos ordenados por cantidad de inscripciones e ingresos generados.

**RC-07.** Tasa de finalización por curso: porcentaje de estudiantes inscritos que completan el curso, cuántos abandonaron en cada etapa.

**RC-08.** **Lección de mayor abandono** por curso: identificar la lección donde la mayor proporción de estudiantes deja de avanzar. Métrica clave para mejorar contenido.



**RC-09.** Tiempo promedio para completar un curso: días entre inscripción y obtención del certificado, agrupado por curso y categoría.

**RC-10.** Análisis de cuestionarios: para cada cuestionario, calificación promedio obtenida, cantidad de intentos por estudiante, preguntas con mayor tasa de error.

**RC-11.** Análisis del foro: cantidad de publicaciones por curso, hilos más activos, estudiantes más participativos, tiempo promedio en recibir primera respuesta.

## 5. Operaciones críticas

Las siguientes operaciones involucran múltiples pasos y deben ejecutarse de forma **atómica** (todo o nada). Tu equipo debe implementarlas de manera que un fallo en cualquier paso revierta todos los cambios anteriores.

**OC-01. Inscripción de estudiante a curso.** Al inscribir un estudiante, el sistema debe: (a) validar que el estudiante exista y el curso esté en estado «publicado», (b) validar que el estudiante no esté ya inscrito en ese curso, (c) registrar el pago del estudiante con el precio vigente del curso, (d) calcular la comisión de la plataforma y el monto a acreditar al instructor, (e) crear la inscripción asociando estudiante con curso, (f) registrar la operación en el log de auditoría. Si cualquier paso falla, ningún cambio debe persistir.

**OC-02. Emisión de certificado.** Al solicitar la emisión de un certificado, el sistema debe: (a) validar que el estudiante esté inscrito en el curso, (b) verificar que el avance del estudiante sea del 100% en el curso, (c) verificar que no se haya emitido previamente un certificado para esa inscripción, (d) generar un código único de verificación, (e) registrar el certificado, (f) registrar la operación en el log de auditoría.

## 6. Volumen mínimo de datos de prueba

Tu equipo debe cargar al menos los siguientes volúmenes para que las consultas y reportes muestren resultados significativos:

| Entidad                      | Cantidad mínima                       |
|------------------------------|---------------------------------------|
| Categorías de curso          | 8                                     |
| Instructores                 | 15                                    |
| Estudiantes                  | 200                                   |
| Cursos publicados            | 25 con mezcla de categorías y precios |
| Módulos por curso (promedio) | 8 a 12                                |

| Entidad                                        | Cantidad mínima                          |
|------------------------------------------------|------------------------------------------|
| Lecciones totales                              | 1,000 o más                              |
| Inscripciones                                  | 600 distribuidas en los últimos 12 meses |
| Registros de progreso de lecciones en MongoDB  | 15,000 o más                             |
| Intentos de cuestionarios en MongoDB           | 800 o más                                |
| Publicaciones en foros con respuestas anidadas | 300                                      |
| Reseñas                                        | 200                                      |
| Certificados emitidos                          | 150                                      |

## Opción D — Delivery de comida

### Resumen del escenario

Plataforma de delivery de comida estilo PedidosYa o Uber Eats que conecta restaurantes con clientes a través de repartidores. La persistencia polígota se justifica porque los pedidos, pagos y la asignación de repartidores requieren ACID estricto y manejo de concurrencia, mientras que los menús (estructura variable según tipo de restaurante), el tracking de eventos del pedido y las calificaciones detalladas son ideales para MongoDB.

### 1. Descripción del negocio

Una plataforma digital de delivery de comida conecta a restaurantes con clientes finales utilizando una red de repartidores propios. La plataforma intermedia cada pedido: muestra los restaurantes disponibles al cliente, procesa el cobro, asigna automáticamente un repartidor para llevar la orden y cobra una comisión por el servicio. Hay tres actores principales: **restaurantes** que publican sus menús y reciben pedidos, **clientes** que ordenan a través de la plataforma, y **repartidores** que recogen y entregan los pedidos.

El flujo típico es el siguiente: el cliente abre la aplicación, ve los restaurantes cercanos abiertos, navega el menú de uno de ellos, arma su pedido escogiendo platos y configurando **modificadores u opciones específicas** (tamaño, extras, ingredientes a omitir, observaciones), confirma la dirección de entrega y paga. El restaurante recibe el pedido, lo confirma y comienza a prepararlo. La plataforma busca un repartidor disponible cercano al restaurante y le asigna el pedido. El repartidor recoge la orden, la lleva al cliente y registra la entrega. Finalmente, el cliente puede calificar el restaurante, la comida y al repartidor en distintos aspectos.

Los **menús varían sustancialmente entre restaurantes**: una pizzería maneja tamaños y toppings, una cafetería maneja tipos de leche y siropes, un restaurante de sushi maneja combos con piezas seleccionables. Adicionalmente, cada pedido genera una serie de **eventos de tracking** (recibido, confirmado, en preparación, listo, recogido, en camino, entregado) que deben registrarse con su marca de tiempo y permitir reconstruir la trazabilidad completa de la orden. Las calificaciones también son ricas: el cliente evalúa múltiples aspectos del restaurante (sabor, presentación, empaque), del repartidor (puntualidad, trato, cuidado del pedido) y deja comentarios libres.

La plataforma requiere generar reportes financieros (comisiones cobradas, montos a liquidar a restaurantes y repartidores) y reportes operativos (tiempos de entrega, restaurantes más demandados, zonas con mayor volumen).

### 2. Requerimientos funcionales

**RF-01.** El sistema debe permitir gestionar el catálogo de categorías gastronómicas (pizza, sushi, comida rápida, café, postres, etc.).

**RF-02.** El sistema debe permitir registrar restaurantes con sus datos, ubicación geográfica, categoría principal, comisión negociada, horarios de atención por día de la semana y estado operativo (activo, suspendido).

**RF-03.** El sistema debe permitir registrar repartidores con sus datos personales, tipo de vehículo, estado (disponible, en pedido, fuera de turno) y ubicación actual.

**RF-04.** El sistema debe permitir registrar clientes con sus datos personales y múltiples direcciones de entrega (casa, trabajo, etc.).

**RF-05.** El sistema debe permitir a cada restaurante publicar su menú, estructurado en categorías de platos con sus precios. Cada plato puede tener **modificadores u opciones configurables** (tamaño, ingredientes adicionales, ingredientes a omitir, especificaciones del cliente). **La estructura del menú varía considerablemente entre restaurantes según el tipo de cocina.**

**RF-06.** El sistema debe permitir consultar restaurantes cercanos a una dirección dada, filtrando por categoría gastronómica y mostrando solo restaurantes que estén abiertos en el horario actual.

**RF-07.** El sistema debe permitir al cliente armar un pedido con múltiples ítems del menú, configurando los modificadores de cada uno y agregando instrucciones especiales. El pedido debe asociarse a una dirección de entrega del cliente.

**RF-08.** El sistema debe registrar el pago del pedido, calculando el desglose: subtotal de productos, costo de envío, comisión de la plataforma para el restaurante, propina opcional para el repartidor.

**RF-09.** El sistema debe asignar automáticamente un repartidor disponible al pedido, considerando su cercanía al restaurante. La asignación debe garantizar que un repartidor solo tenga un pedido activo a la vez.

**RF-10.** El sistema debe registrar el ciclo de vida del pedido a través de **eventos de tracking** (recibido, confirmado por restaurante, en preparación, listo, asignado a repartidor, recogido, en camino, entregado, cancelado). Cada evento debe incluir la marca de tiempo y los datos relevantes (ubicación si aplica, motivo si es cancelación, etc.).

**RF-11.** El sistema debe permitir consultar el estado actual y la línea de tiempo completa de cualquier pedido.

**RF-12.** El sistema debe permitir al cliente calificar un pedido entregado, evaluando múltiples aspectos del restaurante (sabor, presentación, empaque, cumplimiento del pedido), del repartidor (puntualidad, trato, cuidado del pedido) y agregando comentarios libres. **Los aspectos calificados pueden variar y crecer con el tiempo.**

**RF-13.** El sistema debe llevar un registro de auditoría sobre las operaciones críticas (creación de pedidos, asignación de repartidor, registro de pagos, etc.).

### 3. Reglas de negocio

**RN-01.** Solo se pueden recibir pedidos en restaurantes que estén activos y dentro de su horario de atención.

**RN-02.** Un repartidor no puede tener más de un pedido activo al mismo tiempo (entendiendo por «activo» cualquier pedido que aún no haya sido entregado o cancelado).

**RN-03.** La dirección de entrega de un pedido debe pertenecer al cliente que realiza el pedido.

**RN-04.** El total del pedido se calcula como: suma de precios de los ítems con sus modificadores, más costo de envío, menos descuentos aplicables. La comisión de la plataforma se calcula sobre el subtotal de productos.

**RN-05.** Solo se puede calificar un pedido cuyo estado sea «entregado».

**RN-06.** Un pedido puede cancelarse únicamente antes de pasar al estado «en preparación». Después, requiere intervención manual y registro de motivo.

**RN-07.** El precio de los ítems en el pedido debe corresponder al precio vigente del menú al momento de crear el pedido (snapshot), aunque después el restaurante cambie sus precios.

**RN-08.** Un repartidor solo puede asignarse a pedidos cuando su estado es «disponible».

**RN-09.** El monto pagado por un pedido debe coincidir con el total calculado del pedido.

### 4. Reportes y consultas requeridas

El sistema debe poder generar las siguientes consultas y reportes. **Tu equipo decidirá qué mecanismo usar para implementar cada uno** (consulta directa, vista normal, vista materializada, función o pipeline de aggregation), justificando su decisión.

**RC-01.** Restaurantes cercanos abiertos: dado un punto geográfico (lat/long) y una distancia máxima, listar los restaurantes activos abiertos a esa hora con su categoría, calificación promedio y tiempo estimado de entrega.

**RC-02.** Pedidos del cliente: lista de pedidos del cliente con su estado actual y total, ordenados por fecha más reciente.

**RC-03.** Pedidos activos del restaurante: pedidos que el restaurante debe preparar o ya está preparando.

**RC-04.** Pedido en curso del repartidor: pedido actualmente asignado a un repartidor con datos del restaurante de recogida y dirección de entrega.

**RC-05.** Cálculo de total de pedido: dado un conjunto de ítems con modificadores y una dirección, calcular el total con desglose.

**RC-06.** Reporte mensual financiero: comisiones cobradas por la plataforma, monto a liquidar a restaurantes y repartidores, agrupado por mes y categoría gastronómica. Reporte gerencial frecuente.

**RC-07.** Top 10 restaurantes por volumen de pedidos del último trimestre, con ingresos generados.

**RC-08.** Top platos más pedidos por restaurante en el último mes, con cantidad y modificadores más populares.

**RC-09.** Tiempo promedio de entrega por zona y por categoría gastronómica, calculado desde el evento «recibido» hasta el evento «entregado».

**RC-10.** Análisis de calificaciones: promedio de cada aspecto de calificación (sabor, presentación, puntualidad del repartidor, etc.) por restaurante y por categoría.

**RC-11.** Patrones de pedidos por hora del día: cantidad de pedidos por franja horaria y por día de la semana, para identificar horas pico.

## 5. Operaciones críticas

Las siguientes operaciones involucran múltiples pasos y deben ejecutarse de forma **atómica** (todo o nada). Tu equipo debe implementarlas de manera que un fallo en cualquier paso revierta todos los cambios anteriores.

**OC-01. Creación de pedido.** Al crear un pedido, el sistema debe: (a) validar que el restaurante esté activo y abierto, (b) validar que todos los ítems existan en el menú vigente del restaurante, (c) validar que la dirección de entrega pertenezca al cliente, (d) calcular el total con todos sus componentes (subtotal, envío, comisión, propina), (e) crear el pedido en estado «recibido» con el detalle de ítems y sus modificadores capturados como snapshot, (f) registrar el pago, (g) crear el documento de tracking en MongoDB con el primer evento «recibido», (h) registrar la operación en el log de auditoría. Si cualquier paso falla, ningún cambio debe persistir.

**OC-02. Asignación de repartidor.** Al asignar un repartidor a un pedido, el sistema debe: (a) validar que el pedido esté en estado «listo», (b) **buscar y bloquear un repartidor disponible cercano al restaurante manejando correctamente la concurrencia** (dos asignaciones simultáneas no deben poder reservar al mismo repartidor), (c) cambiar el estado del repartidor a «en pedido», (d) asociar el repartidor al pedido, (e) cambiar el estado del pedido a «asignado a repartidor», (f) registrar el evento de tracking correspondiente en MongoDB, (g) registrar la operación en el log de auditoría.

## 6. Volumen mínimo de datos de prueba

Tu equipo debe cargar al menos los siguientes volúmenes para que las consultas y reportes muestren resultados significativos:

| Entidad                                   | Cantidad mínima                                                |
|-------------------------------------------|----------------------------------------------------------------|
| Categorías gastronómicas                  | 6                                                              |
| Restaurantes                              | 30 distribuidos en distintas zonas                             |
| Ítems de menú totales (con modificadores) | 1,500 o más                                                    |
| Repartidores                              | 50 con vehículos variados                                      |
| Clientes                                  | 300 con direcciones múltiples                                  |
| Pedidos                                   | 800 distribuidos en los últimos 6 meses, con mezcla de estados |
| Documentos de tracking en MongoDB         | 800 (uno por pedido) con eventos detallados                    |
| Calificaciones en MongoDB                 | 600 (de pedidos entregados)                                    |
| Entradas de log de auditoría              | 1,500 o más                                                    |